

INTRODUCCIÓN AL DESARROLLO WEB CON **JAVA SPRING BOOT**

En este capítulo daremos el salto desde el desarrollo en Java “tradicional” hacia el **ecosistema de Spring Boot**, una de las herramientas más utilizadas actualmente para **crear aplicaciones modernas de forma rápida y estructurada**.

Hasta ahora, hemos construido programas gestionando manualmente muchos aspectos: configuración, dependencias, acceso a datos o creación de interfaces. Con Spring Boot, gran parte de esa complejidad se simplifica gracias a la **configuración automática** y a un enfoque basado en **convenciones**.

Aprenderemos cómo arrancar una aplicación Spring Boot, organizar el código en capas y crear servicios y controladores de forma sencilla. También veremos cómo se gestionan las dependencias, cómo se integran bases de datos y cómo exponer funcionalidades a través de aplicaciones web de tipo SSR (server-side rendering) completamente funcionales.

Más allá de las herramientas concretas, este capítulo nos introduce en **una nueva forma de pensar el desarrollo**: modular, desacoplado y orientado a componentes. El objetivo no es solo que tus aplicaciones funcionen, sino que sean fáciles de mantener, escalar y evolucionar.

Aquí comienza el camino hacia el **desarrollo verdaderamente profesional** con Java.

1. De Java a un framework Java

1.1. Los límites de la programación clásica

Hasta ahora, durante el curso, hemos trabajado con **aplicaciones Java clásicas**, programas que nosotros controlamos prácticamente desde el principio hasta el final.

Normalmente estas aplicaciones tienen una estructura bastante clara: creamos clases que representan los objetos de nuestro dominio, escribimos métodos que implementan la lógica del programa y, en algún punto, existe un método main() que actúa como puerta de entrada. Cuando ejecutamos el programa, la máquina virtual de Java empieza a ejecutar código desde ahí.

A partir de ese punto todo depende de lo que hayamos programado: creamos objetos, llamamos a métodos, abrimos ficheros, accedemos a bases de datos o mostramos información al usuario.

En muchas de las prácticas que hemos hecho, las aplicaciones también tenían una cierta organización interna. Por ejemplo, cuando trabajamos con bases de datos solemos separar el acceso a los datos utilizando clases tipo DAO (Data Access Object). De esta forma, el código que se encarga de hablar con la base de datos queda separado de la lógica del programa y del interfaz de usuario.

En ese tipo de aplicaciones aparecen elementos bastante habituales:

- Clases y objetos que representan el modelo de datos.
- Un método main() que inicia el programa.
- Clases de acceso a datos (DAO) que se encargan de leer y escribir en la base de datos.
- Algún mecanismo de conexión con la base de datos.
- Una interfaz gráfica de usuario o, a veces, una interfaz de texto.

Este tipo de arquitectura funciona perfectamente para muchos programas.

Sin embargo, **cuando queremos construir una aplicación web moderna, el panorama cambia bastante.**

1.2. Qué es un framework y por qué se usan tanto en el desarrollo web

En una aplicación web ya no tenemos un programa que se ejecuta, hace su trabajo y termina. En su lugar, tenemos **un servidor que está permanentemente escuchando peticiones** que llegan desde navegadores, aplicaciones móviles u otros sistemas.

Esto introduce una serie de problemas nuevos que antes no teníamos que resolver, como:

- Gestionar un servidor HTTP, es decir, un proceso que escuche peticiones de red y responda a ellas.
- Definir rutas (o **endpoints**), que indican qué parte del código se ejecuta cuando llega una determinada petición.
- Convertir datos a JSON y viceversa, porque es el formato más habitual para intercambiar información entre sistemas.
- Conectarse a bases de datos y gestionar transacciones.
- Manejar configuración (puertos, conexiones, credenciales, etc.).
- Administrar dependencias entre distintas librerías.
- Gestionar la autenticación y la seguridad, porque, al ser una aplicación expuesta a una red, va a sufrir ataques con un 100% de probabilidad.

Si intentásemos programar todo esto desde cero cada vez que creamos un proyecto, terminaríamos escribiendo una enorme cantidad de código repetitivo que no tiene que ver con el problema concreto que queremos resolver, sino con los componentes comunes de todas las aplicaciones web.

Y aquí es donde entra en juego una idea fundamental del desarrollo de software moderno: **los frameworks**.

Un framework es, básicamente, un conjunto de herramientas, bibliotecas y convenciones que **nos proporcionan una base sobre la que construir nuestras aplicaciones**. En lugar de empezar desde cero, utilizamos esa base ya preparada, que nos suele dar resueltos esos problemas comunes a todas las aplicaciones.

Gracias a un framework, **muchas tareas complejas o repetitivas ya están hechas antes de empezar**: el servidor web, la gestión de rutas, la conexión con la base de datos, la serialización de datos, etc. De esta forma, el programador puede centrarse en lo realmente importante: la lógica de negocio de su aplicación, que sí es diferente según el problema del que se trate.

1.3. Spring Framework

En el ecosistema de Java, uno de los frameworks más importantes es **Spring Framework**. Se trata de una plataforma muy amplia que ofrece herramientas para construir aplicaciones empresariales, aplicaciones web, sistemas distribuidos y muchos otros tipos de software.

Con el paso del tiempo, Spring se ha ido haciendo cada vez más potente... y también más complejo y pesado. Para simplificar su uso, apareció **Spring Boot**, que es una forma moderna y mucho más simple de crear aplicaciones basadas en Spring.

Spring Boot automatiza gran parte de la configuración y permite arrancar un proyecto funcional con muy poco código. En pocos minutos podemos tener una aplicación que ya funciona como servidor web y que puede ofrecer servicios a otros programas.

1.4. Backend, frontend, API web

Si estás metido en el mundillo de la programación pero nunca has hecho desarrollo web, es posible que te suenen estas palabras pero no sepas muy bien lo que significan. Vamos a aclararlas antes de continuar.

- **Frontend**: es la parte que corre en el navegador web (Chrome, Safari, Edge, Firefox...), la que el usuario ve y con la que interactúa: botones, formularios, vistas, animaciones... Es,

por tanto, la interfaz de usuario, pero también puede incluir lógica propia para gestionar datos, estados y eventos.

- **Backend:** es el programa que corre en el servidor web. Ahí reside la lógica de negocio, el acceso a bases de datos, la autenticación y todo lo que no se quiere exponer directamente al cliente. Es el núcleo del sistema, pero separado físicamente del resto del programa y accesible solo a través de la red.
- **API web:** el frontend y el backend se ejecutan en ordenadores distintos, con distintos sistemas operativos y suelen estar programados en lenguajes diferentes. ¿Cómo pueden dialogar entre sí para constituir una sola aplicación de cara al usuario? La respuesta es el API web, una especie de “contrato de comunicación” entre el frontend y el backend.

Un caso muy habitual de API web es el **API REST**, un conjunto predefinido de endpoints HTTP (es decir, de URLs) que el frontend invoca para pedir o enviar datos. En lugar de llamar funciones directamente como harías en un programa monolítico, aquí haces peticiones HTTP (GET, POST, etc.) y recibes respuestas (normalmente en formato JSON), desacoplando completamente quién pide (frontend) de quién responde (backend).

En las siguientes secciones veremos cómo Spring Boot nos permite construir este tipo aplicaciones de tipo Backend de una manera mucho más sencilla de lo que sería posible partiendo únicamente de Java “puro”. Nuestro objetivo, desde luego, no es aprender ahora todos los detalles del framework, sino entender las ideas básicas que hay detrás y ver cómo se construye una pequeña API web utilizando estas herramientas.

2. ¿Qué es Spring Boot?

Spring Boot como una forma simplificada y moderna de utilizar **Spring Framework**. Su objetivo principal es eliminar gran parte de la configuración inicial de Spring y permitirnos disponer de un proyecto funcional en muy poco tiempo.

En lugar de pasar horas preparando el entorno antes de escribir código, con Spring Boot es posible empezar casi inmediatamente a trabajar en la aplicación.

Spring Boot facilita el desarrollo principalmente de tres maneras:

1. **Configura automáticamente muchas cosas.** El framework analiza el proyecto y, a partir de las dependencias que utilizamos, intenta deducir qué configuración es necesaria.
2. **Incluye un servidor web embebido.** Esto significa que no necesitamos instalar ni configurar un servidor externo para ejecutar nuestra aplicación.
3. **Simplifica la gestión de dependencias**, algo que en proyectos Java grandes puede llegar a ser bastante tedioso.

Gracias a estas características, Spring Boot permite crear aplicaciones web y APIs REST de forma muy rápida, con menos código y configuración que en generaciones anteriores de frameworks Java.

Para entender mejor cómo funciona, conviene fijarse en tres ideas fundamentales sobre las que se basa Spring Boot: autoconfiguración, starter dependencies y servidor web embebido.

2.1. Autoconfiguración

Una de las características más importantes de Spring Boot es la **autoconfiguración**.

La idea es sencilla: el framework **intenta detectar automáticamente qué necesita la aplicación** y configura esos componentes por nosotros.

Por ejemplo, si en el proyecto añadimos una dependencia para trabajar con aplicaciones web, Spring Boot asume que necesitaremos un servidor HTTP y configura automáticamente todo lo necesario para que la aplicación pueda recibir peticiones web.

Del mismo modo, si añadimos dependencias para trabajar con bases de datos, Spring Boot prepara la infraestructura necesaria para gestionar conexiones, transacciones y otras tareas relacionadas.

Esto no significa que no se pueda personalizar la configuración, ni que la autoconfiguración siempre funcione a la perfección. En proyectos reales es bastante habitual que algo falle y haya que configurarlo a mano. Pero lo importante es que no es necesario configurar todo desde cero para empezar a trabajar.

2.2. Starter dependencies

Otro concepto muy importante en Spring Boot es el de **starter dependencies**.

En proyectos Java tradicionales, cuando queremos añadir una nueva funcionalidad solemos tener que incluir varias librerías distintas. A veces estas librerías dependen a su vez de otras, lo que puede generar configuraciones bastante complejas que se intentan gestionar con gestores de proyectos como Maven.

Spring Boot simplifica este proceso agrupando **conjuntos de dependencias que suelen utilizarse juntas**. Estos conjuntos se llaman **starters**.

Por ejemplo, existe un starter específico para desarrollar aplicaciones web. Al incluir ese starter en el proyecto, automáticamente se añaden todas las librerías necesarias para construir un backend HTTP utilizando Spring.

En lugar de tener que investigar qué dependencias necesitamos una por una, basta con incluir el starter adecuado y Spring Boot se encarga del resto.

2.3. Servidor web embebido

En las aplicaciones web Java tradicionales era necesario instalar y configurar un servidor de aplicaciones por separado. Uno de los más conocidos es **Tomcat**.

Después de compilar nuestra aplicación, normalmente se generaba un archivo especial (un archivo WAR) que debía desplegarse dentro de ese servidor para poder ejecutarse.

Spring Boot cambia completamente esta forma de trabajar. Las aplicaciones creadas con Spring Boot **incluyen un servidor web embebido dentro del propio programa**. Esto significa que no necesitamos instalar nada adicional.

Cuando ejecutamos la aplicación, el propio programa arranca su servidor web interno y empieza a aceptar peticiones HTTP. Desde el punto de vista del desarrollador, esto hace que trabajar con aplicaciones web sea mucho más sencillo: basta con ejecutar el proyecto como cualquier otra aplicación Java.

Este servidor embebido, que suele ser Tomcat (aunque hay otros), no solo se usa en desarrollo, sino que también, cada vez más, se usa en producción.

2.4. Estructura mínima de un proyecto con Spring Boot

Un proyecto básico de Spring Boot tiene una estructura bastante sencilla.

Dentro del proyecto encontramos dos directorios principales:

- **src/main/java**, que contiene el código Java de la aplicación. Aquí suele existir una **clase principal** que sirve como punto de entrada de la aplicación, de forma similar al `main()` que hemos utilizado hasta ahora. La diferencia es que, en este caso, esa clase no solo inicia nuestro código, sino que también pone en marcha todo el entorno de Spring Boot, incluido el servidor web y los componentes necesarios para la aplicación.
- **src/main/resources**, donde se colocan archivos con las vistas HTML, archivos de configuración y otros recursos, como imágenes o archivos js o css.

A partir de esa estructura básica iremos añadiendo nuestras clases, controladores, servicios y repositorios, que son los elementos que realmente implementarán la lógica del sistema.

En las siguientes secciones veremos cómo se organiza una aplicación Spring Boot y qué papel cumple

cada una de estas piezas.

3. Crear un proyecto Spring Boot

Crear un proyecto nuevo con Spring Boot, como hemos dicho, es muy rápido, y en pocos minutos podemos tener una aplicación lista para ejecutarse.

En esta sección veremos cómo se crea un proyecto Spring Boot típico, qué elementos contiene y cuál es el papel de la clase principal que inicia la aplicación.

3.1 Generación del proyecto con Spring Initializr

La forma más habitual de crear un proyecto Spring Boot es utilizando **Spring Initializr**, una herramienta que permite generar automáticamente la estructura básica de un proyecto. Podemos lanzarla desde la línea de comandos o en su versión online (<https://start.spring.io/>). O, más fácil aún, ejecutarla desde nuestro entorno de desarrollo como **IntelliJ IDEA** o **Visual Studio Code**.

El funcionamiento de Spring Initializr es bastante sencillo: te hace algunas preguntas sobre tu aplicación y genera un proyecto listo para descargar o abrir en el entorno de desarrollo.

Entre los datos que tendremos que indicar están el nombre del proyecto, el grupo o paquete base, la versión de Java y, sobre todo, las **dependencias** que queremos incluir. Estas dependencias determinarán qué tipo de aplicación vamos a construir.

3.2 Dependencias habituales en aplicaciones backend

Para el ejemplo que construiremos más adelante, utilizaremos estas dependencias comunes en aplicaciones backend con Spring Boot (es decir, esto es lo que le diremos a Spring Initializr que vamos a necesitar):

- **Spring Web**: Esta dependencia incluye todo lo necesario para crear aplicaciones web y APIs REST. Añade las herramientas que permiten definir rutas HTTP, recibir peticiones y devolver respuestas en formato JSON.
- **Spring Data JPA**: Esta dependencia proporciona integración con JPA (Java Persistence API), una tecnología que facilita el acceso a bases de datos relacionales. Gracias a Spring Data JPA es posible trabajar con bases de datos utilizando objetos Java, evitando gran parte del código repetitivo asociado a las consultas SQL.
- **H2 o MySQL**: También necesitaremos una base de datos. Para proyectos pequeños o pruebas suele utilizarse H2, una base de datos muy ligera que puede ejecutarse dentro del propio proyecto. En aplicaciones más reales es habitual utilizar sistemas como MySQL u otros gestores de bases de datos relacionales.
- **Thymeleaf**: Fundamental para renderizar el HTML en el lado del servidor y construir una aplicación SSR 100% funcional por sí misma, sin ningún componente adicional en el lado del cliente (ver explicación en el apartado siguiente)
- **Lombok (opcional)**: Lombok es una librería que ayuda a reducir código repetitivo en las clases Java. Por ejemplo, puede generar automáticamente métodos como getters, setters o constructores. No es imprescindible para trabajar con Spring Boot, pero puede resultar útil para simplificar algunas clases.

3.3 ¿SSR o SPA? El motor de plantillas Thymeleaf

Las aplicaciones web puede funcionar de tres formas principalmente:

- **Aplicación SSR** (Server-Side Rendering): el servidor hace todo el trabajo y genera el HTML que se verá en el cliente. Es la forma clásica y más sencilla para aprender.
- **Aplicación SPA** (Single Page Application): el servidor solo envía datos al navegador web, que

se encarga de renderizarlos y “montarlos” para que se vean bien con Javascript o algún framework como Angular, React o Vue. Es la forma que ha estado de moda desde 2015, más o menos, porque resulta más interactiva para el usuario. Pero estas aplicaciones son más difíciles de montar y mantener.

- o **Enfoque híbrido:** en la actualidad, se está imponiendo un enfoque híbrido, en el que la página se “monta” en el servidor pero se renderiza en el cliente.

Durante el segundo curso trabajarás con los tres enfoques pues, de hecho, ese es el propósito principal de segundo curso. Para esta primera aproximación, construiremos una aplicación SSR, que es la manera más fácil de crear una webapp 100% funcional sin complicarnos demasiado (y sigue siendo, además, una solución completamente profesional)

Pues bien, para renderizar el HTML en el servidor necesitamos **Thymeleaf**, un componente de Spring Boot que se encarga de ello. Por eso debes incluirlo como dependencia al crear el proyecto. En cambio, si tu proyecto fuera de tipo SPA, Thymeleaf no te haría falta para nada, pero necesitarías crear un segundo proyecto en paralelo con Angular, React o Vue para poder ver las pantallas de tu aplicación.

3.4 Estructura básica del proyecto

Una vez generado el proyecto, veremos que tiene una estructura bastante clara.

src/main/java

Aquí está **todo el código fuente en Java** de la aplicación: entidades, controladores, servicios, repositorios, etc.

src/main/resources

Aquí se almacenan el resto de recursos: archivos de configuración, plantillas, imágenes o cualquier otro recurso necesario.

Si nuestra aplicación es de tipo SSR (como, de hecho, es nuestro caso), aquí se almacenarán también las **vistas HTML**.

Uno de los archivos más importantes que aparece también aquí es **application.properties**. Se utiliza para definir los parámetros de configuración de la aplicación, como el puerto del servidor web, la conexión con la base de datos u otras propiedades específicas de Spring. Aunque en proyectos sencillos puede tener muy pocas líneas, en aplicaciones más grandes puede llegar a contener bastante configuración.

3.5 La clase principal de la aplicación

Dentro del proyecto encontraremos también una **clase especial** que actúa como **punto de entrada** de la aplicación. Un **ejemplo típico** sería el siguiente:

```
@SpringBootApplication
public class AplicacionApplication {

    public static void main(String[] args) {
        SpringApplication.run(AplicacionApplication.class, args);
    }

}
```

A primera vista puede parecer una clase Java normal con un método main(). Sin embargo, esta clase tiene un papel muy importante. La anotación **@SpringBootApplication** indica a Spring que esta es la clase principal del proyecto y que desde aquí debe iniciarse el entorno de Spring Boot.

Observa que, cuando ejecutamos el método main(), se llama al método **SpringApplication.run(...)**. Este método arranca todo el sistema de Spring Boot: inicia el servidor web embebido, analiza las

clases del proyecto, crea los objetos necesarios y conecta entre sí los distintos componentes de la aplicación.

En otras palabras, Spring arranca la aplicación por nosotros y prepara todo el entorno necesario para que funcione.

3.6 Inversion of Control (IoC)

La **inversión de control** o **IoC** es un comportamiento muy importante en el funcionamiento de Spring. Vamos a explicarla de manera sencilla.

En una aplicación Java tradicional, nosotros solemos crear los objetos manualmente. Por ejemplo:

```
UsuarioDAO dao = new UsuarioDAO();
UsuarioService servicio = new UsuarioService(dao);
```

El propio código decide cuándo y cómo se crean los objetos, y también se encarga de conectarlos entre sí.

En Spring, el enfoque es diferente. En lugar de que el programador cree los objetos manualmente, **es el framework quien se encarga de hacerlo**. Spring analiza las clases del proyecto, crea las instancias necesarias y las conecta automáticamente.

Esto significa que, en lugar de preocuparnos por construir y enlazar todos los componentes de la aplicación, podemos centrarnos en definir qué hace cada parte del sistema. Spring se ocupa del resto. Esto es lo que se conoce como “inversion of control” o IoC: el control sobre qué objetos se crean y en qué orden pasa de manos del programador a manos del framework.

Esta idea, que puede parecer un poco abstracta al principio, es una de las claves del funcionamiento de Spring Boot y de muchos frameworks modernos. En las siguientes secciones veremos cómo se aplica en la práctica cuando empezamos a construir los distintos componentes de nuestra aplicación.

4. Componentes principales de una aplicación Spring

Una vez creado el proyecto y entendido cómo se pone en marcha una aplicación Spring Boot, el siguiente paso es comprender **cómo se organiza el código dentro de la aplicación**.

Aunque Spring permite muchas formas diferentes de estructurar un proyecto, la mayoría de aplicaciones web siguen una **arquitectura bastante clara basada en capas**. Esta organización ayuda a separar responsabilidades y hace que el código sea más fácil de mantener y entender.

En el pequeño proyecto que construiremos más adelante (un CRUD sencillo) utilizaremos una estructura muy habitual en aplicaciones Spring tipo SSR, formada por estos componentes:

- Modelos (Entity, Repository, Service)
- Vistas
- Controladores

Cada uno de estos elementos cumple una función concreta dentro de la aplicación, dentro de lo que se conoce como patrón MVC (Modelo-Vista-Controlador), un patrón de arquitectura muy popular en aplicaciones web.

4.1 Controller: el punto de entrada de las peticiones

Los controllers son las clases que **reciben las peticiones** que llegan desde el exterior de la aplicación.

Cuando un navegador, una aplicación móvil o cualquier otro programa envía una petición HTTP al servidor, Spring necesita saber qué parte del código debe ejecutarse para responder a esa petición. Esa tarea corresponde a los controllers.

Por ejemplo, imaginemos una aplicación que gestiona usuarios. Podríamos tener rutas como:

- **GET /usuarios** (en el navegador veríamos algo como **https://servidor.com/usuarios**)
- **GET /usuarios/3** (en el navegador veríamos algo como **https://servidor.com/usuarios/3**)
- **POST /usuarios** (en el navegador se vería igual que GET /usuarios, pero internamente http diferenciará entre GET y POST)

Cada una de estas peticiones sería gestionada por un método dentro de un controller, porque cada una espera una cosa distinta del servidor:

- **GET /usuarios** espera que el servidor responda con la lista de todos los usuarios
- **GET /usuarios/3** espera que el servidor responda con los datos detallados del usuario con id 3
- **POST /usuarios** espera que el servidor almacene un usuario nuevo (POST debe ir acompañado de los datos de ese usuario nuevo para que el servidor sepa qué tiene que guardar)

Por tanto, el controller no actuará igual si le llega una petición que si le llega otra. Actúa como una especie de “puerta de entrada” al sistema: recibe la petición que sea, procesa los datos que llegan en ella y decide qué código ejecutar.

Sin embargo, el controller no debería contener toda la lógica de la aplicación. Su función principal es coordinar la petición y delegar el trabajo en otras partes del sistema. Ahí entra en juego el siguiente componente.

4.2 Service: la lógica de negocio

La lógica principal de la aplicación suele colocarse en las clases llamadas **services**.

Un servicio contiene las reglas que determinan cómo funciona el sistema. Es lo que solemos llamar el **modelo** en una **arquitectura MVC** (modelo-vista-controlador).

Por ejemplo, un modelo o service puede encargarse de:

- crear un usuario nuevo
- decidir qué datos deben validarse
- decidir qué operaciones se realizan antes de guardar cierta información en la base de datos
- decidir cómo se combinan distintas consultas

El controller recibe la petición HTTP, pero normalmente delega el trabajo real en un servicio.

Por ejemplo, un flujo de ejecución típico podría ser:

- **Vista:** El usuario escribe los datos de un nuevo usuario en un formulario de “alta de usuarios” (vista) y pulsa el botón “Enviar”. Los datos del formulario se envían por POST al servidor.
- **Controlador:** El controller recibe la petición POST /usuario para crear un usuario nuevo (junto con los datos del usuario) y llama a un método del servicio (o modelo) que se encargará realmente de crear el usuario.
- **Modelo:** El servicio (modelo) ejecuta la lógica necesaria para crear el usuario, incluyendo el acceso a la base de datos para guardar los datos.

Separar esta lógica en una capa de servicios ayuda a mantener el código organizado y permite reutilizar esa lógica desde distintos lugares si fuera necesario.

4.3 Repository: acceso a los datos

El service no accede directamente a la base de datos, sino que existe otra capa encargada de ello: la capa **repository**.

Su función es **leer y escribir información en las tablas de la base de datos**. En otras palabras, es la

parte de la aplicación que se comunica directamente con el sistema de persistencia. Es muy similar al **ORM** que ya hemos utilizado en nuestras aplicaciones (Hibernate/JPA).

Spring Boot generará automáticamente todas las operaciones necesarias para persistir y consultar nuestros datos, como *findAll()*, *findById()*, *save()* o *delete()*, lo que reduce considerablemente la cantidad de código que necesitamos escribir.

4.4 Entity: representación de los datos

El último componente del **modelo** es la entidad o **Entity**.

Una entity es **una clase que representa una tabla de la base de datos**. Como hacíamos en nuestras aplicaciones Java con Hibernate/JPA, vamos.

Por ejemplo, si tenemos una tabla llamada usuarios, solemos tener también una clase Java llamada Usuario. Los atributos de esa clase se corresponden con las columnas de la tabla.

Estas clases permiten trabajar con los datos de la base de datos utilizando objetos Java en lugar de manipular directamente filas y columnas.

En aplicaciones que utilizan JPA y frameworks como Hibernate, las entities suelen estar asociadas a tablas mediante anotaciones que indican cómo se relacionan los campos de la clase con las columnas de la base de datos.

4.5 Vistas: ¿cómo se muestra todo esto?

Cuando el controlador recibe una petición (por ejemplo, GET /usuarios, es decir, “dame todos los usuarios”), lo normal es que le pida al modelo la lista de usuarios.

Pero, una vez que obtiene un List<Usuario>, ¿qué hace con esos datos? El controlador tiene dos opciones:

- **Enviárselos al cliente** (navegador web) en forma de datos planos, normalmente formateados en JSON. Esto es lo que se hace en las aplicaciones SPA, donde correcta la visualización de los datos se delega al navegador web.
- **Generar una vista HTML** con esos datos y devolverla al navegador web lista para mostrarla. Así, el navegador no tiene que hacer nada, salvo recibir un HTML ya construido y mostrárselo al usuario final.

Nosotros, en este primer ejemplo, vamos a hacer esto último, es decir, **generar un HTML con los datos**. Y aquí es donde entran en juego las **vistas**, que son páginas web escritas con **HTML entremezclado con Java**, de manera que las variables Java pueden usarse para crear ese HTML.

Para lograr este efecto, Spring Boot utiliza un **motor de plantillas** llamado **Thymeleaf** que veremos en funcionamiento un poco más adelante.

4.6 Cómo trabajan juntas todas las capas

Cuando una aplicación Spring Boot está en funcionamiento, las capas suelen colaborar siguiendo un flujo bastante claro.

De forma simplificada, el proceso suele ser algo así:

1. **Vista:** el usuario interactúa con la vista y lanza una petición al servidor.
2. **Controlador:** El controller recibe una petición HTTP y decide a qué método del service hay que llamar.
3. **Modelo (I):** El service ejecuta la lógica necesaria.
4. **Modelo (II):** El service llama al repository para leer o escribir datos.
5. **Modelo (III):** El repository accede a la base de datos utilizando las entities.

6. **Controlador:** El controller recibe los datos del Modelo y se los pasa a una vista
7. **Vista:** La vista genera el código HTML con los datos solicitados y los envía al navegador web, de manera que el usuario final puede verlos.

Esta separación en capas permite organizar el código de forma más clara, facilita las pruebas y hace que cada parte del sistema tenga una responsabilidad bien definida.

En el ejemplo práctico que veremos más adelante utilizaremos exactamente esta estructura. A través de un pequeño CRUD podremos observar cómo cada uno de estos componentes participa en el funcionamiento de una aplicación Spring Boot sencilla.

Una última aclaración: recuerda que **la capa "vista" podría no existir**. Depende de si estamos construyendo una aplicación web tipo SSR (server-side rendering) o tipo SPA (single page application), o incluso un API, que ni siquiera tiene interfaz de usuario y solo recibe y envía datos a otras aplicaciones.

(Estas arquitecturas más complejas de aplicaciones las trabajarás el curso que viene).

5. Anotaciones fundamentales

Una de las características más llamativas de Spring Boot es que gran parte de su funcionamiento se basa en **anotaciones**.

Las anotaciones son elementos del lenguaje Java que permiten **añadir información adicional a clases, métodos o atributos**. En el caso de Spring, estas anotaciones indican al framework **qué papel cumple cada componente** dentro de la aplicación.

Cuando Spring Boot arranca, **analiza el código del proyecto y busca estas anotaciones**. A partir de ellas decide qué clases debe crear, cómo deben conectarse entre sí y cómo deben responder a las peticiones que recibe la aplicación.

Existen muchas anotaciones dentro del ecosistema Spring, pero para entender cómo funciona una aplicación básica solo necesitamos conocer unas pocas. A continuación veremos las más importantes.

5.1 @SpringBootApplication

Esta anotación aparece en la **clase principal del proyecto**, la que contiene el método **main()** que inicia la aplicación.

```
@SpringBootApplication
public class MiAplicacion {
    public static void main(String[] args) {
        SpringApplication.run(MiAplicacion.class, args);
    }
}
```

Es decir, la anotación **@SpringBootApplication** indica a Spring Boot que esta clase es el **punto de inicio** del sistema.

Cuando ejecutamos la aplicación, Spring Boot utiliza esta anotación para comenzar a analizar el proyecto, buscar componentes y preparar todo el entorno necesario para que la aplicación funcione. Esto incluye, entre otras cosas:

- iniciar el servidor web embebido
- localizar los controllers utilizados
- crear los servicios y repositorios
- configurar la aplicación

5.2 @Controller y @RestController

Un **controller**, como ya hemos explicado, es una clase que se encarga de **recibir peticiones HTTP**. En Spring Boot, este tipo de clases pueden marcarse con la anotación **@Controller** o con **@RestController**.

```
@Controller
public class UsuarioController {
}
```

- Si usas **@Controller**, estás informando a Spring de que ese es un controlador genérico que responde a cualquier endpoint y que devuelve páginas web completas, renderizadas como HTML válido.
- Si usas **@RestController**, estás informando a Spring de que ese controlador implementa un API REST y, por tanto, solo responde a algunos endpoints y devuelve datos crudos en JSON o XML.

5.3 @RequestMapping

La anotación **@RequestMapping** se utiliza para definir la ruta o URL asociada a un controller o a uno de sus métodos.

Por ejemplo:

```
@RequestMapping("/usuarios")
```

Esto indica que el controller o método asociado responderá a peticiones que empiecen por la URL **/usuarios**.

De esta forma podemos organizar nuestra API en diferentes rutas que representen los recursos de la aplicación.

Aunque **@RequestMapping** es bastante flexible, en la práctica suele combinarse con anotaciones más específicas que indican el tipo de operación HTTP que se está realizando (GET, POST, etc).

5.4 @GetMapping, @PostMapping, @PutMapping y @DeleteMapping

Estas anotaciones se utilizan para asociar métodos de una clase controller con operaciones HTTP concretas.

Cada una corresponde a uno de los verbos HTTP más habituales:

@GetMapping se utiliza para obtener información (petición HTTP tipo GET)

@PostMapping se utiliza para crear nuevos recursos (petición HTTP tipo POST)

@PutMapping se utiliza para actualizar información existente (petición HTTP tipo PUT)

@DeleteMapping se utiliza para eliminar recursos (petición HTTP tipo DELETE)

Por ejemplo:

```
@GetMapping("/usuarios")
public void miMetodo() { ... }
```

Esto significa que el método `miMetodo()` se ejecutará cuando el servidor reciba una petición de tipo HTTP GET en la ruta o URL **/usuarios**.

5.5 @Autowired

Otra anotación muy importante es **@Autowired**. Se utiliza para indicar que **una clase necesita utilizar otro componente del sistema**. En lugar de crear ese objeto manualmente, dejamos que Spring se encargue de proporcionarlo.

Por ejemplo:

```
@Autowired  
UsuarioService usrService;
```

Aquí estamos indicando que esta clase necesita un objeto de tipo `UsuarioService`. No necesitamos hacer `usrService = new UsuarioService()` ni nada parecido, como en Java tradicional, sino que `@Autowired` se encarga de hacerlo por nosotros y de inyectar ese objeto en la clase que lo necesita.

Este proceso se conoce como **inyección de dependencias** y es una consecuencia directa del principio de **Inversión de Control (IoC)** que vimos anteriormente.

5.6 @Entity, @Table, @Column

La anotación **@Entity** se utiliza, como ya vimos, en las clases que representan **tablas de la base de datos**. A veces va acompañada de la anotación **@Table**, para indicar a Spring el nombre exacto de la tabla (si no, Spring tratará de inferir ese nombre a partir del nombre de la clase)

Por ejemplo, para una tabla de usuarios:

```
@Entity  
@Table(name = "usuarios")  
public class Usuario {  
    private Long id;  
    private String nombre, apellido1, apellido2, apellido3;  
    private String email;  
    private LocalDate fechaNacimiento;  
  
    // Constructor por defecto (requerido por JPA)  
    public Usuario() {}  
  
    // Luego irían getters, setters y cualquier otro método necesario  
}
```

Los atributos de la clase suelen corresponder con las **columnas de la tabla**. Esto permite trabajar con los datos de la base de datos utilizando objetos Java, lo que simplifica bastante el código.

Dentro de la clase marcada como **@Entity** pueden aparecer muchas otras anotaciones para especificar propiedades de las columnas. Por ejemplo:

@Id marca qué campo ejerce de clave primaria de la tabla

@GeneratedValue indica que ese campo se genera automáticamente. Suele usarse con las claves primarias (el típico `AUTOINCREMENT` de MySQL)

@Column se utiliza para mapear ese campo con una columna de la tabla e indicar si esta admite nulos, si es o no `UNIQUE`, etc.

Teniendo esto presente, podemos mostrar un ejemplo completo de la **@Entity usuario**:

```
import jakarta.persistence.*;  
import java.time.LocalDate;  
  
@Entity  
@Table(name = "usuarios")  
public class Usuario {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id; // autoincrementable  
  
    @Column(nullable = false)  
    private String nombre;  
  
    @Column(nullable = false)  
    private String apellido1;  
  
    @Column(nullable = false)  
    private String apellido2;
```

```

@Column(nullable = false, unique = true)
private String email;

@Column(name = "fecha_nacimiento", nullable = true)
private LocalDate fechaNacimiento; // Puede ser null

// Constructor por defecto (obligatorio para JPA)
public Usuario() {}

// Constructor conveniente para crear objetos
public Usuario(String nombre, String apellido1, String apellido2,
                String email, LocalDate fechaNacimiento) {
    this.nombre = nombre;
    this.apellido1 = apellido1;
    this.apellido2 = apellido2;
    this.email = email;
    this.fechaNacimiento = fechaNacimiento;
}

// Getters y setters
public Long getId() {
    return id;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido1() {
    return apellido1;
}

public void setApellido1(String apellido1) {
    this.apellido1 = apellido1;
}

public String getApellido2() {
    return apellido2;
}

public void setApellido2(String apellido2) {
    this.apellido2 = apellido2;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public LocalDate getFechaNacimiento() {
    return fechaNacimiento;
}

public void setFechaNacimiento(LocalDate fechaNacimiento) {
    this.fechaNacimiento = fechaNacimiento;
}
}

```

5.7 @Repository

Las clases o interfaces que se encargan del **acceso a datos** suelen marcarse con la anotación **@Repository**.

```

@Repository
public interface UsuarioRepository {
}

```

Un repository, como ya dijimos, es **la capa que se comunica directamente con la base de datos**. Su función es realizar operaciones como buscar registros, guardar datos, actualizar información o eliminar registros.

Es decir, como vimos anteriormente, el papel del repository es muy parecido al de las **clases DAO** que ya hemos utilizado en otros proyectos con bases de datos.

5.8 @Service

Por último, tenemos la anotación **@Service**, que se utiliza para marcar las clases que contienen la **lógica de negocio** de la aplicación (modelos).

```
@Service
public class UsuarioService {
}
```

Los servicios actúan como **intermediarios entre los controllers y los repositorios**. Reciben las solicitudes del controller, aplican la lógica necesaria y utilizan los repositorios para acceder a la base de datos si la operación lo requiere.

6. Configuración básica de Spring Boot

Una de sus características más apreciadas de Spring Boot es la enorme cantidad de **configuración automática** que realiza. De hecho, ese es uno de los objetivos de los framework.

Aun así, hay ciertos parámetros básicos que tienes que definir explícitamente. Muchos de ellos se almacenan juntos, en un archivo de configuración llamado **application.properties**.

En este archivo se definen cosas como qué base de datos debe conectarse la aplicación, qué puerto usar, cómo gestionar las tablas, etc.

Vamos a ver los parámetros imprescindibles que tenemos que configurar en **application.properties** para que una aplicación Spring Boot, como el API REST que acabamos de crear, pueda conectarse a una base de datos y trabajar con ella.

6.1 El archivo application.properties

El archivo **application.properties** se guarda en **src/main/resources**. Si lo abres, te encontrarás con un montón de parámetros definidos como **parejas clave = valor**. Algo así:

```
spring.datasource.url=jdbc:mysql://localhost/test
spring.datasource.username=root
spring.datasource.password=1234
spring.jpa.hibernate.ddl-auto=update
```

- **spring.datasource** indica a Spring Boot cómo conectarse a la base de datos
 - **spring.datasource.url=jdbc:mysql://localhost/test** → define la dirección de la base de datos (tipo de base de datos, servidor y nombre de la base de datos)
 - **spring.datasource.username=root** → define el nombre del usuario de la base de datos
 - **spring.datasource.password=1234** → define... bueno, ya te lo imaginas, ¿no?
- **spring.jpa.hibernate.ddl-auto=update** permite que Spring, con Hibernate, genere automáticamente las tablas de la base de datos, si no existen.

Hibernate tomará como referencia las Entities que hayamos definido y creará las tablas a partir de ahí. El valor "update" implica que, si hacemos un cambio en una Entity, Hibernate modificará

también la estructura de la tabla. Por lo tanto:

- si la tabla no existe → Hibernate la crea
- si añadimos un campo nuevo → Hibernate añade la columna
- si modificamos la entidad → Hibernate intenta adaptar la tabla

Esto resulta extremadamente útil durante el desarrollo, ya que evita tener que crear manualmente las tablas cada vez que cambiamos el modelo de datos.

ddl-auto admite otros valores distintos de “update”, como:

- **none** → No realiza cambios en la base de datos
- **validate** → Comprueba que la estructura coincide
- **update** → Actualiza la estructura si es necesario
- **create** → Elimina y vuelve a crear las tablas
- **create-drop** → Crea las tablas al iniciar y las borra al cerrar

Durante el desarrollo suele utilizarse update o create. En entornos de producción (cuando una aplicación ya está online y funcionando), normalmente se emplea none o validate.

7. Acceso a datos con Spring Data JPA

Hasta ahora, hemos visto diferentes formas de acceder a una base de datos desde Java:

- Utilizando JDBC directamente (método tradicional)
- Empleando patrones como DAO (método mejorado)
- Apoyándonos en herramientas ORM como Hibernate (método moderno)

En todos los casos, aparece siempre la misma idea: **necesitamos que una parte del programa se encargue de comunicarse con la base de datos**. Cuando trabajamos con Spring Boot, esa parte es el **Spring Data JPA**.

Spring Data JPA es una extensión del ecosistema Spring que **facilita enormemente el acceso a bases de datos** relacionales **utilizando JPA** (Java Persistence API). Su principal objetivo es reducir al mínimo el código necesario para realizar operaciones comunes con la base de datos. Spring Data JPA trabaja con Hibernate y lo hace aún más sencillo de usar.

Spring Data JPA no es Hibernate, pero se parece bastante. De hecho, es una evolución de Hibernate, así que todo te sonará bastante familiar.

7.1 Cómo se hacía tradicionalmente

Si repasas cómo funcionaban nuestras clases DAO, verás que, para acceder a la base de datos, había que realizar varias tareas repetitivas. Por ejemplo, para una consulta SELECT típica era necesario:

1. Escribir la consulta SQL que queríamos ejecutar.
2. Abrir una conexión con la base de datos.
3. Crear un PreparedStatement y ejecutar la consulta.
4. Recorrer el ResultSet para convertir los resultados en objetos Java (tal vez ArrayList)
5. Cerrar los recursos utilizados.

Incluso si utilizamos frameworks ORM, seguimos escribiendo bastante código repetitivo para gestionar estas operaciones.

En proyectos pequeños esto puede ser manejable, pero en aplicaciones más grandes genera mucho código repetitivo, lo que aumenta la complejidad del sistema y dificulta inútilmente su mantenimiento.

7.2 La idea de Spring Data JPA

Spring Data JPA intenta resolver este problema con una idea bastante simple: **generar automáticamente gran parte del código de acceso a datos**.

En lugar de escribir clases completas que implementen las operaciones básicas, el programador solo necesita definir una interfaz que represente el repositorio de una entidad.

Por ejemplo, para una **tabla usuarios**, definiríamos un interface **UsuarioRepository**:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {  
}
```

- Usuario es la Entity en la que se quiere implementar el interface. ¡No tienes que hacer nada más! Spring Boot SPA se encargará de implementarlo por ti.
- Long es el tipo de datos de la clave primara de Usuario.

A primera vista puede parecer una tontería sorprendentemente simple. No hay métodos, no hay consultas SQL, no hay código que abra conexiones con la base de datos. ¿Para qué narices sirve definir ese interface?

Sencillo: a partir de ese simple interface, **Spring Data JPA generará una implementación completa del código de acceso a datos para la entidad Usuario** y, por tanto, para la tabla usuarios con la que está mapeada.

A partir de ahora, en la clase Usuario tienes disponibles un montón de métodos para acceso a datos. Los vemos a continuación.

7.3 Operaciones que se generan automáticamente

Cuando una interfaz hace **extends JpaRepository**, Spring crea automáticamente una serie de métodos que permiten realizar muchas operaciones comunes sobre la base de datos. Entre ellos, están estos métodos:

- save() → guarda un objeto en la base de datos
- findAll() → devuelve todos los registros
- findById() → busca un registro por su identificador
- delete() → elimina un registro

¿Te suenan? Son los típicos métodos que, con esos nombres u otros parecidos, generábamos en nuestros objetos DAO en Java tradicional. Ahora Spring Data JPA los genera para nosotros y no tenemos que escribirlos.

Por ejemplo, en nuestro service UsuarioService, podremos escribir llamadas como estas y funcionarán a la perfección:

```
usuarioRepository.save(usuario);  
List<Usuario> usuarios = usuarioRepository.findAll();
```

Spring generará internamente las instrucciones SQL necesarias, abrirá y cerrará conexiones y, en definitiva, gestionará la interacción con la base de datos. El programador puede desentenderse de todos esos detalles repetitivos y centrarse en la lógica de su problema.

8. Generación de vistas con Thymeleaf

Las vistas, como ya hemos dicho, son los documentos HTML que la aplicación web genera a partir de las solicitudes del usuario.

No todas las aplicaciones web generan vistas. Solo lo hacen las aplicaciones tipo SSR (server-side rendering). Otras aplicaciones web, como los APIs, generan datos crudos que serán renderizados en el lado del cliente.

8.1 Plantillas

Las vistas de Spring Boot se generan a partir de documentos .html algo modificados, porque permiten incrustar dentro código Thymeleaf, que enseguida veremos lo que es.

Esos documentos .html manipulados se denominan **plantillas o templates** y se almacenan en *src/main/resources/templates*

8.2. Cómo es un template o plantilla Thymeleaf

Lo mejor es verlo con un ejemplo. Observa el siguiente template *usuariosList.html*, escrito para generar un HTML con la lista de todos los registros de la tabla “usuarios”:

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Lista de Usuarios</title>
  <meta charset="UTF-8">
</head>
<body>

<h1>Lista de Usuarios</h1>

<table border="1">
  <thead>
    <tr>
      <th>ID</th>
      <th>Nombre</th>
      <th>Email</th>
      <th>Password</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="usuario : ${usuarios}">
      <td th:text="${usuario.id}"></td>
      <td th:text="${usuario.nombre}"></td>
      <td th:text="${usuario.email}"></td>
      <td th:text="${usuario.password}"></td>
    </tr>
  </tbody>
</table>

</body>
</html>
```

Parece HTML convencional, ¿verdad? Pero, si miras el código con atención, verás algo raro, algo que no es HTML convencional.

Mira la etiqueta <tr> de la tabla. ¿Qué narices es ese **th:each** y la expresión **\${usuarios}** que aparece después?

Eso es un **atributo Thymeleaf**, el motor de plantillas de Spring Boot para generar HTML a partir de datos preparados por los controladores. Cuando veas “th:” en un HTML, ahí está Thymeleaf trabajando.

Como parece obvio, el atributo **th:each** provoca una iteración de tipo for-each sobre un conjunto de datos (una colección). Y esa colección es **\${usuarios}**. Es decir, **\${usuarios}** es una variable de Thymeleaf y, de algún modo, se ha colocado un array de usuarios hay dentro de esa variable para que Thymeleaf lo recorra (enseguida veremos quién ha colocado ahí ese array).

El resto de elementos <td> tienen un **th:text** con el que se accede a cada uno de los miembros de cada usuario, generándose así los elementos de la tabla.

8.3 Qué puede hacer Thymeleaf

Los atributos de Thymeleaf tienen un montón de utilidades en las vistas. Estas son las más importantes:

1. Para mostrar cualquier dato inyectado desde el controlador

```
<p th:text="${usuario.nombre}"></p>
```

2. Para iterar sobre colecciones, es decir, recorrer listas (muy típico en tablas):

```
<tr th:each="usuario : ${usuarios}">
```

3. Para mostrar u ocultar contenido condicional:

```
<p th:if="${usuario.activo}">Activo</p>
```

4. Para integrarse en formularios. Se puede establecer el valor del *action* o el contenido de un *input* (el asterisco * indica que, si {nombre} tiene un valor, debe usarse como value del input y, si no, el input se dejará en blanco)

```
<form th:action="@{/usuarios}" method="post">
  <input type="text" th:field="*{nombre}" />
</form>
```

5. Para generar enlaces dinámicos y no tener que escribir direcciones absolutas.

```
<a th:href="@{/usuarios/{id}(id=${usuario.id})}">Ver</a>
```

6. Para reutilizar otras plantillas, como menús de navegación, headers, footers, para evitar escribirlas repetidas muchas veces.

```
<div th:replace="fragments/header :: header"></div>
```

Thymeleaf puede hacer muchas más cosas, pero con esas, de momento, vamos bien servidos.

8.5 ¿Y de dónde vienen los datos?

En el ejemplo anterior, el template de Thymeleaf iteraba sobre una variable \${usuarios}. ¿Quién rellenó esa variable con la lista de usuarios?

La respuesta es simple: los datos los preparara el **controlador**.

Recuerda que los controladores capturan las peticiones del navegador web en forma de solicitudes HTTP (como "GET /usuarios"), y ejecuta un método concreto como respuesta a esa solicitud.

Lo que debe hacer el controlador de usuarios como respuesta a "GET /usuarios" es:

1. Pedirle al modelo (servicio) de Usuario que recupere la lista con todos los usuarios (el controlador nunca trabaja directamente con la base de datos)
2. Pasar esa lista a la vista usuariosList para que genere un HTML con los datos.

Un controlador que haga exactamente eso podría tener este aspecto:

```
@Controller
@RequestMapping("/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }
}
```

```

    }

    @GetMapping("")
    public String listarUsuarios(Model model) {
        model.addAttribute("usuarios", usuarioService.obtenerTodos());
        return "usuariosList"; // Nombre de la vista (thymeleaf)
    }
}

```

En el código anterior:

- **@RequestMapping("/usuarios")** hace que este controlador se ejecute cuando se pida la URL "/usuarios"
- **@GetMapping("")** hace que el método **listarUsuarios()** se ejecute cuando no se haya añadido nada más a la URL "/usuarios" y la petición HTTP venga por GET. Esto es, cuando se haya pedido "GET /usuarios".

Observa bien lo que hace el método **listarUsuarios()**:

- Añade a la variable "model" un atributo llamado "usuarios" cuyo contenido es lo que ha devuelto obtenerTodos().
- Hace un return "usuariosList", que es el nombre de la plantilla Thymeleaf que hemos visto antes. Spring Boot buscará un archivo src/main/resources/usuariosList.html y tratará de cargarlo, enviándole una variable llamada "usuarios" que contendrá toda la lista de usuarios.

Para que esto funcione, obviamente, el Service de usuarios debe tener escrito un método llamado obtenerTodos(), más o menos así:

```

@Service
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    public List<Usuario> obtenerTodos() {
        return usuarioRepository.findAll();
    }
}

```

Como ves, el Service recurre, a su vez, al Repository para acceder a los datos, pero el método findAll() del repository no hace falta programarlo, puesto que Spring Boot lo genera para nosotros.

9. Creación de una aplicación web CRUD completa

Con todo lo que hemos visto, ya estamos en condiciones de crear nuestra primera aplicación web 100% funcional con Spring Boot. Verlo todo bien reunido hará que todos los conceptos que hemos recorrido poco a poco queden mucho más claros.

Vamos a construir un **ejemplo sencillo para hacer un CRUD de una tabla de usuarios**.

Para ello, lanzaremos **Spring Initializr** desde nuestro IDE preferido (como Visual Studio Code) y seleccionaremos las dependencias mínimas que ya hemos mencionado antes:

- **Spring Web**
- **Spring Data JPA**
- **H2** (para pruebas rápidas) **o MySQL**. Ojo, si eliges MySQL, tendrás que poner en marcha un servidor MySQL adicional, puesto que Spring no trae ninguno embebido.
- **Thymeleaf**

9.1 Operaciones CRUD

Te recuerdo que el acrónimo CRUD se refiere al ciclo de operaciones típicas con cualquier recurso de una base de datos: create-read-update-delete.

Si queremos construir un CRUD de usuarios, estamos hablando concretamente de:

- **Create** → Crear datos: la aplicación web nos permitirá introducir (en un formulario) los datos de un nuevo usuario y lo guardará en la base de datos.
- **Read** → Leer datos: la aplicación debe ser capaz de mostrarnos el listado de todos los usuarios y también los datos detallados de un usuario concreto.
- **Update** → Actualizar datos: la aplicación nos permitirá modificar los datos de un usuario ya existente y debe guardar los cambios en la base de datos.
- **Delete** → Eliminar datos: la aplicación debe permitirnos eliminar un usuario de la base de datos.

9.2 CRUD de usuarios como aplicación web SSR

Sin más preámbulo, vamos a crear una pequeña webapp SSR (server-side rendering) para gestionar usuarios. Supondremos que cada usuario tiene solo tres campos:

- id
- nombre
- email

Vamos a construir una aplicación Spring Boot típica, con el código organizado en estas capas:

- *Entity* → representa la tabla de la base de datos (usuarios)
- *Repository* → acceso a datos
- *Service* → lógica de negocio
- *Controller* → captura las peticiones y llama a los servicios adecuados
- *Vistas* → renderiza la salida HTML

9.3 La entidad Usuario (Entity)

Una **Entity**, como sabemos, representa una de las **tablas** que se almacenan en la base de datos. En nuestro caso, como trabajamos con usuarios, nuestra Entity es una **class Usuario** con este aspecto:

```
import jakarta.persistence.*;

@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String email;

    public Usuario() {}

    public Usuario(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
    }

    public Long getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }
}
```

```

public String getEmail() {
    return email;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public void setEmail(String email) {
    this.email = email;
}
}

```

9.4 El repositorio (Repository)

El **Repository**, como ya hemos explicado, es la capa que se encarga de **acceder a la base de datos**.

El repositorio para usuarios se crea simplemente **extendiendo la interfaz JpaRepository**.

```

import org.springframework.data.jpa.repository.JpaRepository;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
}

```

¡Y eso es todo! **Spring genera automáticamente** los métodos `findAll()`, `findById()`, `save()`, `deleteById()` y otros muchos para la tabla de usuarios.

9.5 La capa de servicio (Service)

Un **Service** contiene la **lógica de negocio** de la aplicación. Es decir, es nuestro **modelo**. Aquí se establece cómo se gestionan los usuarios. Observa que nuestro código se limita a invocar métodos del Repository, sin hacer realmente nada nuevo. Muchos Services son así de simples.

```

import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UsuarioService {

    private final UsuarioRepository repository;

    public UsuarioService(UsuarioRepository repository) {
        this.repository = repository;
    }

    public List<Usuario> obtenerTodos() {
        return repository.findAll();
    }

    public Usuario obtenerPorId(Long id) {
        // Si el id no existe, devolverá null
        return repository.findById(id).orElse(null);
    }

    public Usuario guardar(Usuario usuario) {
        return repository.save(usuario);
    }

    public void eliminar(Long id) {
        repository.deleteById(id);
    }
}

```

9.6 El controlador (Controller)

El **Controller** la capa que expone los endpoints de la aplicación, es decir, el lugar donde se enlazan los endpoints (como “GET /usuario/3” o cualquier otro endpoint) con el código Java.

El controller usará, por un lado, el Service de usuarios (para acceder a los datos) y, por otro, las vistas (para generar las salidas HTML).

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/usuarios")
public class UsuarioController {

    private final UsuarioService service;

    public UsuarioController(UsuarioService service) {
        this.service = service;
    }

    // LISTAR USUARIOS
    @GetMapping("")
    public String listarUsuarios(Model model) {
        model.addAttribute("usuarios", service.obtenerTodos());
        return "usuarioList";
    }

    // MOSTRAR UN USUARIO
    @GetMapping("/{id}")
    public String verDetalle(@PathVariable Long id, Model model) {
        Usuario usuario = service.obtenerPorId(id);

        if (usuario == null) {
            return "redirect:/usuarios";
        }

        model.addAttribute("usuario", usuario);
        return "usuarioDetalle";
    }

    // FORMULARIO CREAR
    @GetMapping("/nuevo")
    public String mostrarFormularioNuevo(Model model) {
        model.addAttribute("usuario", new Usuario());
        return "usuarioForm";
    }

    // GUARDAR (CREATE y UPDATE)
    @PostMapping("/guardar")
    public String guardarUsuario(@ModelAttribute Usuario usuario) {
        service.guardar(usuario);
        return "redirect:/usuarios";
    }

    // FORMULARIO EDITAR
    @GetMapping("/editar/{id}")
    public String mostrarFormularioEditar(@PathVariable Long id, Model model) {
        Usuario usuario = service.obtenerPorId(id);
        model.addAttribute("usuario", usuario);
        return "usuarioForm";
    }

    // ELIMINAR
    @GetMapping("/eliminar/{id}")
    public String eliminarUsuario(@PathVariable Long id) {
        service.eliminar(id);
        return "redirect:/usuarios";
    }
}
```

Observa, en el código anterior, estas anotaciones importantes:

@Controller: Indica a Spring Boot que esta clase define un controlador.

@RequestMapping("/usuarios"): Define la ruta base de todos los métodos de este controlador.

@GetMapping, @PostMapping: Asocian métodos HTTP con métodos Java. Fíjate que algunos llevan un parámetro, como `@GetMapping("/{id}")`. Eso significa que el endpoint incluirá un dato

“id”, no solo el verbo HTTP y la ruta.

Por ejemplo, en el caso de `@GetMapping("/{id}")`, significa que la petición HTTP debe tener esta forma:

GET /usuario/id

El dato “id” de ese endpoint se usará para pasárselo al método `verDetalle()` del controlador. Por eso, si pedimos el endpoint GET /usuario/3, el servidor sabe que tiene que consultar el usuario con id = 3 y mostrarnos una vista con su detalle

@PathVariable: Permite leer variables que se han pasado por la URL, como el “id = 3” de GET /usuario/3.

@ModelAttribute: Permite recibir los datos de un formulario HTML empaquetados (mapeados) en un objeto de tipo Usuario

Con el código de este controlador, ya tenemos construida una sencilla aplicación CRUD para la tabla de usuarios, con estos endpoints:

- GET /usuarios: Obtener una vista HTML con todos los usuarios
- GET /usuarios/{id}: Obtener una vista HTML con un usuario
- GET /nuevo: Mostrar una vista HTML con un formulario para creación de un nuevo usuario
- POST /usuarios: Crear un nuevo usuario o modificar uno existente
- GET /usuarios/editar/{id}: Mostrar una vista HTML con un formulario para modificar un usuario que ya existe (por eso se le pasa un id)
- GET /usuarios/eliminar/{id}: Eliminar el usuario con ese id. Después de eliminar, redirigirá a GET /usuarios para volver a mostrar la lista de usuarios.

9.7 Las vistas

El controlador anterior usa 3 vistas:

- **usuariosList:** muestra la lista de usuarios. Añadiremos a esta vista varios links, para añadir usuarios nuevos (GET /usuarios/nuevo), modificar un usuario (GET /usuarios/editar/{id}), eliminar usuarios (GET /usuarios/eliminar/{id}) y ver el detalle de un usuario (GET /usuarios/{id}). Por lo tanto, ejercerá como vista principal de nuestra aplicación, porque desde aquí se puede hacer todo.
- **usuariosForm:** muestra un formulario para añadir un usuario nuevo. Reutilizaremos la misma vista para que nos permita editar un usuario existente.
- **usuariosDetalle:** muestra los datos de un solo usuario

Vamos a ver el código de cada una de ellas.

Vista `src/main/resources/templates/usuarioList.html`

Esta, como te he dicho, es la vista principal, pues incluye toda la información y links para ejecutar el resto de acciones. Observa bien cómo los links no se crean con direcciones absolutas, sino con expresiones de Thymeleaf como **`th:href="@{/usuarios/nuevo}"`**

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Lista de usuarios</title>
  <meta charset="UTF-8">
</head>
<body>

<h1>Lista de usuarios</h1>

<a th:href="@{/usuarios/nuevo}">Añadir usuario</a>

<br><br>

<table border="1">
```

```

<thead>
  <tr>
    <th>ID</th>
    <th>Nombre</th>
    <th>Email</th>
    <th>Acciones</th>
  </tr>
</thead>
<tbody>
  <tr th:each="usuario : ${usuarios}">
    <td th:text="${usuario.id}"></td>

    <!-- Si hacemos click en el nombre nos lleva a la vista detalle -->
    <td>
      <a th:href="@{/usuarios/{id}(id=${usuario.id})}"
        th:text="${usuario.nombre}">
      </a>
    </td>

    <td th:text="${usuario.email}"></td>

    <td>
      <a th:href="@{/usuarios/editar/{id}(id=${usuario.id})}">Editar</a>
      |
      <a th:href="@{/usuarios/eliminar/{id}(id=${usuario.id})}"
        onclick="return confirm('¿Seguro que quieres eliminar este usuario?')">
        Eliminar
      </a>
    </td>
  </tr>
</tbody>
</table>

</body>
</html>

```

Vista src/main/resources/templates/usuarioForm.html

Como esta vista debe servirnos como formulario para añadir y para modificar usuarios, verás que el título se muestra con un condicional de tipo (condición?acción-true:acción-false). La “condición” de esa expresión consiste en mirar si existe un id de usuario asignado. Si es así, estamos tratando de modificar un usuario. Si no existe un id, es porque estamos tratando de añadir un usuario nuevo.

Observa también cómo tratan de rellenarse los inputs del formulario con el contenido del usuario. El asterisco en *th:field="*{nombre}"* le dice a Thymeleaf que, si existe un valor para “nombre”, debe mostrarlo en ese input y, si no existe, debe dejar el input en blanco.

Ese sencillo truco nos permite reutilizar el mismo formulario para añadir usuarios nuevos (el formulario saldrá en blanco) y para modificar usuarios existentes (el formulario saldrá relleno con los datos actuales del usuario).

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title th:text="${usuario.id} == null ? 'Nuevo usuario' : 'Editar usuario'"></title>
  <meta charset="UTF-8">
</head>
<body>

<h1 th:text="${usuario.id} == null ? 'Crear usuario' : 'Editar usuario'"></h1>

<form th:action="@{/usuarios/guardar}" th:object="${usuario}" method="post">

  <!-- ID oculto para edición -->
  <input type="hidden" th:field="*{id}" />

  <div>
    <label>Nombre:</label><br>
    <input type="text" th:field="*{nombre}" required />
  </div>

  <br>

```

```

<div>
  <label>Email:</label><br>
  <input type="email" th:field="*{email}" required />
</div>

<br>

<button type="submit">Guardar</button>
</form>

<br>

<a th:href="@{/usuarios}">Volver al listado</a>

</body>
</html>

```

Vista `src/main/resources/templates/usuarioDetalle.html`

Esta vista muestra el detalle de un usuario, un link para volver a la vista principal y otro link para modificar este usuario.

Es la vista más simple de las tres y no tiene demasiado sentido, porque los usuarios de esta pequeña aplicación apenas tienen datos. Sin embargo, en una aplicación real, donde un usuario pudiera tener muchos más campos, sería una vista importante para poder visualizar todos los campos del usuario que no se vieran en la lista de usuarios.

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Detalle de usuario</title>
  <meta charset="UTF-8">
</head>
<body>

<h1>Detalle del usuario</h1>

<p>
  <strong>ID:</strong>
  <span th:text="${usuario.id}"></span>
</p>

<p>
  <strong>Nombre:</strong>
  <span th:text="${usuario.nombre}"></span>
</p>

<p>
  <strong>Email:</strong>
  <span th:text="${usuario.email}"></span>
</p>

<br>

<a th:href="@{/usuarios}">Volver al listado</a>
|
<a th:href="@{/usuarios/editar/{id}(id=${usuario.id})}">Editar</a>

</body>
</html>

```

9.8 Probando la aplicación

Para probar la aplicación, solo nos queda configurarla en `src/main/resources/application.properties`, donde tenemos que añadir los datos de la conexión a la BD. Si elegimos MySQL como base de datos, esos datos serán:

```

spring.datasource.url=jdbc:mysql://localhost:3306/tu_database
spring.datasource.username=tu_usuario
spring.datasource.password=tu_password
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.hibernate.ddl-auto=update

```

¡OJO! Tendrás que poner en marcha el servidor de MySQL por tu cuenta. Spring Boot tiene un servidor web embebido, pero no un servidor de MySQL.

Una vez hecho esto, ya tenemos una mini aplicación web terminada, lista para interactuar con un humano a través de un navegador web. Solo tienes que abrir tu navegador web preferido y probar el endpoint principal:

http://localhost:8080/usuarios

Esa URL te dará acceso al endpoint “GET /usuarios”, que te mostrará la lista de usuarios y links al resto de endpoints a los que responde el controlador de usuarios (añadir, modificar, borrar, etc)

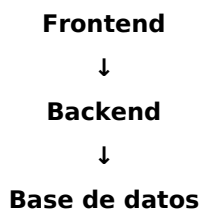
10. Y ahora, ¿qué?

A lo largo de este pequeño recorrido hemos visto una **primera aproximación a Spring Boot**, y a las aplicaciones web de backend de tipo SSR, es decir, que generan sus propias vistas en el servidor.

Nuestro objetivo no es dominar Spring Boot en profundidad ni volvernos expertos en desarrollo web (jeso solo ocurrirá con los años y la experiencia!), sino abrir una pequeña ventana a cómo funcionan las aplicaciones web modernas. El próximo curso, si estás estudiando Desarrollo de Aplicaciones Web, ampliarás todo lo que hemos visto (aunque seguirás sin ser “un experto”).

La estructura típica de los proyectos Spring Boot (Entities, Repositories, Services, Controllers, Vistas) es muy semejante a la de otros frameworks para backend, como Laravel (que usa PHP en lugar de Java), Django (que usa Python) o Nest (que usa Javascript). Hay pequeñas diferencias de nomenclatura y organización, pero la esencia es la misma, así que todo lo que has aprendido te resultará muy útil para profundizar en esos y otros frameworks.

Sin embargo, para hacer desarrollo web moderno aún te faltan varios componentes de la arquitectura web habitual. Normalmente, una aplicación web se divide en tres grandes bloques:



En esta introducción a Spring Boot hemos trabajado en el lado del **backend** y de la **base de datos**. Es el mismo lugar donde trabajan **Laravel** (PHP), **Django** (Python) o **Nest** (Javascript).

El **frontend** es la **interfaz que ve el usuario y que se ejecuta en el navegador web o en la aplicación móvil**. Casi siempre está escrito en **Javascript** o **Typescript** (un Javascript supervitaminado), o, más en concreto, está escrito con algún framework para Javascript como **Angular**, **React**, **Vue**, **Next** o **Svelte**. Estos frameworks se encargan de construir interfaces dinámicas y aplicaciones web muy interactivas, descargando al programador del trabajo más tedioso, como hacen los frameworks de backend, pero en el frontend.

El trabajo en segundo curso de Desarrollo de Aplicaciones Web consiste, sobre todo, en aprender a utilizar todas (o gran parte) de esas tecnologías para construir aplicaciones web completas y siguiendo los estándares modernos de desarrollo.

De momento, mi consejo es que practiques con Spring Boot para construir aplicaciones SSR sencillas que generen sus propias vistas HTML. Si esto te sabe a poco y quieres más, échale un vistazo a alguna de las tecnologías del lado front, como React o Vue, para empezar a crear tus primeros frontend que consuman los datos generados desde Spring Boot.

Ejercicios Apéndice I

Prácticas finales

Academia de idiomas: gestión de cursos y estudiantes

Vamos a crear una **pequeña aplicación web SSR** (server-sider rendering) con Spring Boot que **gestione los estudiantes y cursos** de una **academia de idiomas**.

Los alumnos de la academia se podrán inscribir en cursos, y podremos consultar, crear, actualizar y eliminar tanto estudiantes como cursos mediante el típico CRUD.

Aunque el ejercicio sea sencillo, toca muchos conceptos importantes de Spring Boot: entidades, relaciones, repositorios, servicios, controladores, vistas y endpoints HTTP.

Objetivos

1. Crear dos entidades relacionadas: Estudiante y Curso (en los apuntes solo hemos visto cómo crear una entidad. Crear dos es igual, pero tendrás que investigar cómo relacionar una con otra)
2. Aprender a crear relaciones de muchos a muchos (un estudiante puede estar en varios cursos, y un curso puede tener varios estudiantes).
3. Construir repositorios JPA para acceder a los datos de cada tabla.
4. Implementar la lógica de negocio en servicios.
5. Exponer endpoints para cada entidad.
6. Construir las vistas de entrada de datos y de salida de datos (form/list)
7. Probar la aplicación.

PASO 0. Crear un proyecto Spring Boot

Crea un proyecto Spring Boot nuevo con Visual Studio Code. Utiliza **Spring Initializr**, la herramienta que permite generar automáticamente la estructura básica del proyecto.

(Si es la primera vez, es posible que necesites instalar el plugin de Spring Initializr en tu VS Code)

Durante la creación del proyecto tendrás que responder a varias preguntas: versión de Spring (da un poco igual para lo que vamos a hacer), lenguaje base (elige Java), nombre del paquete, etc.

IMPORTANTE: cuando te pregunte qué dependencias quieres instalar, asegúrate de marcar:

- Spring Web
- Spring Data JPA
- MySQL (H2 es más ligero, pero solo sirve para pruebas y sus datos se destruyen al terminar. Usaremos MySQL o MariaDB, que son viejos conocidos)
- Thymeleaf

PASO 1. Definición de las tablas y creación de entidades

Tabla estudiantes (entidad Estudiante)

- id → clave primaria, autoincrementable
- nombre → no nulo
- apellido → no nulo
- email → no nulo, único
- fechaNacimiento → puede ser nula
- relación con cursos → muchos a muchos

Tabla cursos (entidad Curso)

- id → clave primaria, autoincrementable
- nombre → no nulo
- descripcion → puede ser nula
- relación con estudiantes → muchos a muchos

Relación cursos-estudiantes: se trata de una relación N:N (muchos a muchos), porque un curso puede tener muchos estudiantes y cada estudiante puede estar matriculado en varios cursos.

PISTA: Investiga sobre la estrategia **@ManyToMany** para almacenar la relación N:N entre cursos y estudiantes. Hibernate generará automáticamente la tabla intermedia.

PASO 2. Repositorios

Crea dos repositorios que extiendan JpaRepository:

- EstudianteRepository
- CursoRepository

Con esto Spring Boot te proporcionará automáticamente los métodos `findAll()`, `findById()`, `save()` y `deleteById()`, entre otros, para cada entidad.

PASO 3. Servicios

Cada entidad tendrá su service para encapsular la “lógica de negocio”, es decir, la lógica de esa entidad.

El servicio **EstudianteService** debe permitir:

- Obtener todos los estudiantes
- Obtener un estudiante por id
- Crear o actualizar un estudiante
- Eliminar un estudiante
- Inscribir un estudiante en un curso
- Retirar un estudiante de un curso

El servicio **CursoService** hará algo similar para los cursos.

PASO 4. Controladores

En este paso vamos a crear los controladores que se encargarán de recibir las peticiones HTTP, interactuar con los servicios y devolver las vistas con Thymeleaf.

Crearemos dos controladores, y cada uno gestionará las operaciones CRUD con una entidad, así como las relaciones entre entidades.

- EstudianteController
- CursoController

4.A) EstudianteController

Este controlador manejará todas las operaciones relacionadas con los estudiantes. Atenderá estos endpoints:

GET /estudiantes: Muestra la lista de todos los estudiantes.

GET /estudiantes/{id}: Muestra el detalle de un estudiante concreto (incluyendo sus cursos).

GET /estudiantes/nuevo: Muestra el formulario para crear un nuevo estudiante.

POST /estudiantes: Procesa el formulario y guarda un nuevo estudiante.

GET /estudiantes/editar/{id}: Muestra el formulario para editar un estudiante existente.
POST /estudiantes/{id}: Actualiza los datos de un estudiante.
GET /estudiantes/eliminar/{id}: Elimina un estudiante y redirige al listado.
POST /estudiantes/{estudianteld}/cursos/{cursold}: Inscribe un estudiante en un curso.
POST /estudiantes/{estudianteld}/cursos/{cursold}/eliminar: Retira un estudiante de un curso.

4.B) CursoController

Este controlador gestionará todas las operaciones relacionadas con los cursos, con estos endpoints:

GET /cursos: Muestra la lista de todos los cursos.
GET /cursos/{id}: Muestra el detalle de un curso (incluyendo sus estudiantes).
GET /cursos/nuevo: Muestra el formulario para crear un nuevo curso.
POST /cursos: Procesa el formulario y guarda un nuevo curso.
GET /cursos/editar/{id}: Muestra el formulario para editar un curso existente.
POST /cursos/{id}: Actualiza los datos de un curso.
GET /cursos/eliminar/{id}: Elimina un curso y redirige al listado.

Notas importantes:

- Los controladores deben usar anotaciones como @Controller y @RequestMapping. Para manejar formularios, se usarán @ModelAttribute. Para parámetros en la URL, se utilizará @PathVariable. Revisa el ejemplo resuelto de los apuntes para tener algo en lo que basarte. ¡No le pidas el código resuelto a una IA! Funcionará, pero no habrás aprendido nada.
- Tras operaciones POST, es recomendable hacer redirect (vuelve a mirar el código de ejemplo)
- Envía datos a las vistas usando el objeto Model.

PASO 5. Crear las vistas

En este paso vamos a crear las 6 vistas que necesita esta aplicación, utilizando Thymeleaf. Estas vistas permitirán al usuario interactuar con la aplicación desde el navegador, mostrando datos y proporcionando formularios para crear y editar entidades.

5.A) Vistas de Estudiantes

Vista 1. estudiantes/list.html: Muestra una tabla con todos los estudiantes, con links para Ver detalle, Editar, Eliminar, Añadir

Vista 2. estudiantes/detail.html: Muestra la información completa de un estudiante (nombre, apellido, email, fecha de nacimiento, lista de cursos en los que está inscrito). También tendrá un link para inscribir al estudiante en un nuevo curso y otro para eliminar inscripción de un curso

Vista 3. estudiantes/form.html: Formulario para crear o editar un estudiante. El mismo formulario servirá tanto para crear como para editar.

5.B) Vistas de Cursos

Vista 4. cursos/list.html: Muestra una tabla con todos los cursos, más links para Ver detalle, Editar, Eliminar y Crear nuevo.

Vista 5. cursos/detail.html: Muestra la información completa de un curso (nombre, descripción, lista de estudiantes inscritos)

Vista 6. cursos/form.html: Formulario para crear o editar un curso.

PASO 6. Configuración de la aplicación

Debes editar *application.properties* para configurar los parámetros mínimos de la aplicación:

```
spring.datasource.url=jdbc:mysql://localhost:3306/mi_db
spring.datasource.username=tu_usuario
spring.datasource.password=tu_password
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.hibernate.ddl-auto=update
```

PASO 7. Probar la aplicación

Para ejecutar la aplicación, desde Visual Studio Code, localiza la clase principal del proyecto (la que contiene **@SpringBootApplication** y el método `main()`) y ejecútala. VSCode debió escribirla para ti al crear el proyecto, y su contenido debe ser algo así:

```
@SpringBootApplication
public class AcademiaApplication { // Puedes ponerle otro nombre
    public static void main(String[] args) {
        SpringApplication.run(AcademiaApplication.class, args);
    }
}
```

¡OJO! Si has elegido MySQL o MariaDB como base de datos, **tendrás que poner en marcha el servidor de MySQL por tu cuenta**. Spring Boot tiene un servidor web embebido, pero no un servidor de MySQL.

Si todo está correcto, el servidor arrancará, normalmente en el puerto 8080. Abre tu navegador y entra en la dirección:

<http://localhost:8080/estudiantes>

o en:

<http://localhost:8080/cursos>